Semestrálny projekt NI-PDP 2025/2026:

Paralelný algoritmus pre riešenie problému quatromín s minimom

Adam Kleman

magisterské štúdium, FIT ČVUT, Thákurova 9, 160 00 Praha 6

11. apríla 2026

1 Skladanie quatromín s minimom a jeho sekvenčná implementácia

Úlohou je nájsť pokrytie obdĺžnikovej hracej dosky D quatrominami tvaru T a Z s minimálnou reziduálnou cenou. Vstupom je obdĺžniková doska rozmerov $a \times b$, kde $3 \leq a, b \leq 20$, pričom platí $a \cdot b \geq 15$. Každé políčko dosky je ohodnotené prirodzeným číslom z intervalu $(0, 100)$. Jednotlivé quatrominá sa nesmú prekrývať a musia sa zmestiť do hraníc dosky. Musí byť splnená podmienka, že rozdiel quatromina typu T a Z sa nesmie líšiť o viac než 1. Výsledná cena je súčet všetkých nepokrytých políčok, takže cieľom je mať dosku so splnenými podmienkami a výsledným súčtom čo najmenším.

Sekvenčná verzia algoritmu je implementovaná ako prehľadávanie stavového priestoru metódou Branch-and-Bound s použitím prehľadávania do hĺbky pomocou DFS. Každý stav reprezentuje čiastočné pokrytie dosky použitými tvarmi T a Z.

Doska je reprezentovaná ako jednorozmerné pole veľkosti $rows \cdot cols$. Hodnoty v poli majú nasledovný význam: hodnota 0 označuje nerozhodnuté políčko, kladné hodnoty reprezentujú umiestnené quatrominá typu T, záporné hodnoty quatrominá typu Z a špeciálna hodnota označuje nepokryté políčko, ktorého hodnota sa započítava do ceny riešenia.

Algoritmus začína vygenerovaním množiny počiatočných konfigurácií, kde je na prázdnu dosku umiestnené práve jedno quatromino (T alebo Z) vo všetkých možných pozíciách a orientáciách. Pre každú takúto konfiguráciu sa následne spustí rekurzívne prehľadávanie. V každom kroku prehľadávania sa vyberie prvé nerozhodnuté políčko, čím sa zabezpečí systematické prechádzanie stavového priestoru od „ľavého horného“ k „pravému spodnému“ políčku hracej dosky.

Pre každú vetvu sa vytvorí nový stav a prehľadávanie pokračuje rekurzívne do hĺbky. Po dosiahnutí úplného riešenia (t. j. keď už neexistujú nerozhodnuté políčka) sa overí podmienka vyváženosti počtu quatromín typu T a Z. Ak je riešenie prípustné a jeho cena je nižšia ako doteraz najlepšie riešenie, aktualizuje sa globálne optimum.

Efektivita algoritmu je zabezpečená pomocou viacerých prerezávacích podmienok (branch-and-bound). Základnou je dolná medza ceny riešenia, ktorá sa počíta ako súčet najmenších možných hodnôt políčok, ktoré musia zostať nepokryté vzhľadom na to, že quatrominá pokrývajú vždy štyri políčka. Ak súčet aktuálnej ceny a tejto dolnej medze presiahne alebo sa rovná doteraz najlepšiemu riešeniu, vetva je zrezaná.

Ďalšia podmienka prerezávania vychádza z obmedzenia na počet použitých quatromín typu T a Z. Ak rozdiel medzi ich počtom už nie je možné dorovnať vzhľadom na zostávajúci počet políčok, vetva je ukončená. Algoritmus taktiež využíva možnosť predčasného ukončenia výpočtu v prípade, že sa nájde riešenie s cenou rovnou triviálnej dolnej medze, ktorá predstavuje teoretické minimum.

2 Implementácia táskového paralelizmu pomocou OpenMP

Tasková paralelná verzia vychádza priamo zo sekvenčného algoritmu BB-DFS a zachováva rovnakú reprezentáciu stavov aj rovnaké prerezávacie podmienky, teda každý stav prehľadávaného priestoru obsahuje stav dosky.

Hlavná zmena oproti sekvenčnej implementácii spočíva v tom, že jednotlivé vetvy rekurzívneho prehľadávania sú v horných úrovniach stromu spracovávané ako samostatné OpenMP tasky. Vstupný stav je v tejto verzii prázdna doska a prehľadávanie sa spúšťa z jedného koreňového stavu v rámci paralelnej oblasti, ktorú začína direktíva `parallel`. Jedno vlákno typu `master` inicializuje výpočet a vytvára tasky, ktoré následne vykonávajú ostatné vlákna tímu. Počet vlákien je možné nastaviť parametrom príkazového riadku `-t` a maximálna hĺbka, do

ktorej sa ešte vytvárajú nové tasky, je riadená parametrom `-d`. Parameter `-d` sa na zadaných úlohách osvedčil pre úlohy do hĺbky 4.

Prehľadávanie je rozšírené o parameter hĺbky a pokiaľ je hĺbka menšia než zvolený limit, každé vytvorené potomok je spracované ako nový OpenMP task. Po dosiahnutí tejto hranice sa ďalšie vetvy už nerobia ako tasky, ale spracujú sa sekvenčne. Na konci spracovania daného uzla sa používa direktíva `taskwait`, aby sa dokončilo spracovanie všetkých potomkov vytvorených v tomto uzle.

Keďže všetky tasky zdieľajú globálne najlepšie riešenie, je nutné synchronizovať prístup k zdieľaným premenným. Počet rekurzívnych volaní sa aktualizuje atomicky. Premenné `stop` a `bestCost` sa najprv čítajú pomocou atómických operácií do lokálnych kópií, aby sa znížil počet synchronizovaných prístupov. V prípade nájdenia nového najlepšieho riešenia sa používa kritická sekcia, v ktorej sa bezpečne aktualizuje riešenie, teda hodnoty `bestCost`, `bestState`, `bestT` a `bestZ`. Pri tvorení taskov sa každý nový stav odovzdáva ako `firstprivate`, ostatné zostávajú ako zdieľané premenné.

3 Implementácia dátového paralelizmu pomocou OpenMP

Dátová paralelná verzia algoritmu taktiež vychádza zo sekvenčného riešenia, pričom sa paralelizuje výpočet z nezávislých počiatočných konfigurácií. Každá počiatočná konfigurácia predstavuje stav, v ktorom je na dosku umiestnené jedno quatromino, a následné prehľadávanie stavového priestoru prebieha nezávisle pomocou BB-DFS pre každú z nich. Na začiatku výpočtu sa vygeneruje množina všetkých možných počiatočných konfigurácií, ktoré sa ukladajú do poľa `std::vector`, čo predstavuje frontu, z ktorej sú prideľované počiatočné konfigurácie vláknam. Každá konfigurácia obsahuje aktuálny stav dosky, počet použitých quatromín typu T a Z a aktuálnu cenu riešenia.

Paralelizácia je realizovaná pomocou OpenMP direktívy `parallel`, pričom jedno vlákno (master) inicializuje výpočet a vytvára jednotlivé konfigurácie. Tieto úlohy sú následne spracovávané ostatnými vláknami, pričom každé vlákno vykonáva sekvenčný BB-DFS algoritmus nad pridelenou konfiguráciou a hľadá lokálne najlepšie riešenie. Po dosiahnutí najlepšieho riešenia v danej vetve, je nutná aktualizácia globálneho riešenia, ktorá je realizovaná pomocou kritickej sekcie, aby nedošlo ku konfliktu medzi vláknami.

Keďže jednotlivé konfigurácie sú navzájom nezávislé, väčšina výpočtu prebieha bez potreby synchronizácie. Synchronizácia je však nevyhnutná pri práci so zdieľanými premennými, ktoré reprezentujú globálne najlepšie riešenie. Zdieľané premenné `bestCost` a príznak `stop` sú čítané pomocou atómických operácií, aby sa zabezpečila konzistentnosť medzi vláknami. V prípade nájdenia nového lepšieho riešenia sa používa kritická sekcia, v ktorej sa bezpečne aktualizuje globálny stav riešenia. Využíva sa aj atómická operácia pri aktualizácii počtu rekurzívnych volaní do lokálnej kópie. Rovnako ako v taskovej verzii algoritmus využíva zdieľanú hornú medzu ceny riešenia, ktorá sa priebežne aktualizuje a využíva na prerezávanie neperspektívnych vetiev. Táto hodnota je zdieľaná medzi všetkými vláknami a prispieva k zníženiu počtu prehľadávaných stavov.

4 Implementácia kombinácie dátového paralelismu pomocou OpenMP a MPI pre prácu s distribuovanou pamäťou

MPI verzia algoritmu je implementovaná ako hybridné riešenie kombinujúce distribuovaný paralelizmus medzi procesmi a paralelizmus so zdieľanou pamäťou pomocou OpenMP vo vnútri každého procesu. Základ algoritmu opäť vychádza zo sekvenčného BB-DFS, avšak stavový priestor je rozdeľovaný medzi viaceré MPI procesy v architektúre master-slave.

Úlohou procesu s rankom 0 je najprv vytvoriť počiatočnú frontu stavov pomocou obmedzeného BFS rozvoja do zadanej hĺbky. Táto hĺbka je riadená parametrom `-l`. Výsledkom nie je jedno riešenie, ale množina čiastočných konfigurácií, ktoré predstavujú korene nezávislých podstromov ďalšieho prehľadávania. Tieto konfigurácie master rozdeľuje slave procesom po dávkach, pričom veľkosť dávky je riadená parametrom `-c`.

Slave procesy pracujú v opakovanej slučke prijímania práce od master procesu. V aktívnom stave slave prijme chunk konfigurácií, prevezme od mastera aj aktuálnu hodnotu najlepšej známej ceny a následne tento chunk spracuje lokálne pomocou funkcie `solveQueueOpenMP`. Táto funkcia využíva v rámci jedného procesu OpenMP

paralelizmus nad množinou pridelených konfigurácií, takže každý slave je sám o sebe viacvláknový. Po dokončení chunku worker odošle masterovi svoje lokálne najlepšie riešenie, počet rekurzívnych volaní a počet použitých vlákien, inak slave čaká na ďalšiu správu od mastera. Ak prijme špeciálnu správu s tagom ukončenia, opustí pracovnú slučku a ukončí sa.

Master proces po počiatočnom rozoslaní práce priebežne prijíma výsledky od ľubovoľného slave procesu. Po prijatí výsledku aktualizuje globálne najlepšie riešenie a rozhodne, či danému workeru pošle ďalší chunk práce, alebo mu odošle ukončovaciu správu.

Pre rozoslanie práce sa odosiela hlavička obsahujúca počet konfigurácií v chunku a aktuálnu najlepšiu známu cenu riešenia. Komunikácia medzi procesmi je realizovaná explicitne pomocou MPI správ typu point-to-point. Používajú sa viaceré typy správ rozlíšené pomocou 6 custom tagov, napríklad správy pre rozoslanie práce (WORK), prenos samotného stavu dosky (BOARD), odoslanie výsledkov (RESULT) a ukončenie výpočtu (STOP). Každá správa obsahuje buď riadiace informácie (napr. veľkosť chunku, aktuálnu hornú medzu ceny), alebo samotné dáta reprezentujúce konfiguráciu dosky. Tento prístup umožňuje flexibilné riadenie výpočtu a dynamické prideľovanie práce medzi procesmi.

Algoritmus využíva aj viaceré parametre, napríklad parameter `-t` sa nastavuje počet OpenMP vlákien v jednom procese, parametrom `-l` hĺbka BFS rozvoja a parametrom `-c` veľkosť chunku odosielaného jednemu slave procesu.

5 Namerané výsledky a vyhodnotenie

5.1 Nastavenie experimentov

Merania boli realizované na klastri `cluster-arm.fit.cvut.cz` pomocou plánovača úloh SLURM. Pre OpenMP implementácie bol počet vlákien riadený parametrom `-t` a nadobúdal hodnoty 1, 2, 4, 8, 12, 16, 32 a 48. MPI implementácia bola spúšťaná s 4 procesmi (1 master a 3 slave procesy). Každý proces využíval viacvláknový výpočet pomocou OpenMP. Pre MPI verziu bol počet vlákien v procese nastavený podľa najlepších výsledkov získaných z OpenMP meraní.

Čas merania predstavuje čistý čas výpočtu bez započítania načítania vstupných dát a výstupu výsledkov. Meranie bolo realizované pomocou funkcií `omp_get_wtime()` a `MPI_Wtime()`. Každá konfigurácia bola spustená viackrát a výsledný čas predstavuje priemernú hodnotu z týchto behov. Namerané výsledky sú uvedené v Tabuľke 1 a Tabuľke 2. Merania boli realizované pre tri rôzne vstupné inštancie rôznej náročnosti.

Zrýchlenie S bolo vypočítané vzhľadom na implementovanú sekvenčnú verziu algoritmu. Efektivita paralelného výpočtu bola počítaná ako zrýchlenie vzhľadom na sekvenčný čas a p predstavuje počet použitých výpočtových jadier. V prípade OpenMP implementácie zodpovedá p počtu vlákien, zatiaľ čo pri hybridnej MPI a OpenMP implementácii bolo uvažované $p = 4 \cdot t$, kde 4 je počet MPI procesov a t počet vlákien v rámci jedného procesu.

$$S = \frac{T_{\text{seq}}}{T_t} \qquad\qquad E = \frac{S}{p}$$

5.2 Meranie Task a Data paralelizmu

Zo získaných výsledkov je zrejmé, že tasková paralelizácia dosahuje výrazne lepšie zrýchlenie než dátová paralelizácia na danej vzorke inštancií. Pri zvyšovaní počtu vlákien dochádza k výraznému poklesu výpočtového času, pričom najlepšie výsledky boli dosiahnuté pre vyššie hodnoty parametra `t` (napr. 32 alebo 48 vlákien). Zlepšenie však nie je lineárne a pri vyšších počtoch vlákien sa prejavuje saturácia výkonu spôsobená režijnými nákladmi na správu taskov a synchronizáciu.

Typ	počet jadier (`-t`)	hĺbka (`-d`)	mapa5_15			mapa15_5			mapa9_9		
			Čas [s]	S	E	Čas [s]	S	E	Čas [s]	S	E
Sekv.	1	-	170.857	1.00	1.00	197.231	1.00	1.00	462.494	1.00	1.00
Task	1	4	46.265	3.69	3.69	135.118	1.46	1.46	23.070	20.05	20.05
	2	4	36.099	4.73	2.37	11.851	16.64	8.32	26.177	17.67	8.83
	4	4	9.898	17.26	4.32	4.428	44.54	11.14	7.018	65.90	16.48
	8	4	4.618	37.00	4.62	3.428	57.54	7.19	3.805	121.55	15.19
	12	4	3.365	50.77	4.23	1.966	100.32	8.36	3.006	153.86	12.82
	16	4	3.696	46.23	2.89	2.103	93.79	5.86	3.226	143.36	8.96
	32	4	2.933	58.25	1.82	1.825	108.07	3.38	2.686	172.19	5.38
	48	4	1.268	134.75	2.81	1.580	124.83	2.60	2.946	156.99	3.27
Data	1	-	157.316	1.09	1.09	190.034	1.04	1.04	441.536	1.05	1.05
	2	-	91.619	1.86	0.93	104.524	1.89	0.94	223.755	2.07	1.03
	4	-	45.103	3.79	0.95	47.904	4.12	1.03	112.598	4.11	1.03
	8	-	24.787	6.89	0.86	23.189	8.51	1.06	58.173	7.95	0.99
	12	-	17.751	9.63	0.80	16.873	11.69	0.97	40.204	11.50	0.96
	16	-	14.051	12.16	0.76	12.341	15.98	1.00	31.766	14.56	0.91
	32	-	8.277	20.64	0.65	7.171	27.50	0.86	17.003	27.20	0.85
	48	-	6.162	27.73	0.58	4.985	39.56	0.82	12.998	35.58	0.74

Tabuľka 1: Porovnanie priemerných časov behov, zrýchlenia a efektivity pre sekvenčnú verziu a OpenMP implementácie.

Dátová paralelizácia vykazuje stabilné zrýchlenie, avšak výrazne pomalšie než tasková verzia. Hodnoty parametrov `-d` a `-l` boli zvolené na základe experimentálneho ladenia OpenMP implementácií.

Výsledky ukazujú, že efektivita paralelizácie závisí aj od konkrétnej inštancie problému. Napríklad pre inštanciu `mapa9_9` je zrýchlenie výraznejšie než pre ostatné prípady, čo naznačuje väčší a vhodnejšie štruktúrovaný stavový priestor pre paralelné spracovanie.

Naopak, pri niektorých konfiguráciách sa objavujú výkyvy v čase, čo je spôsobené nevyváženým rozdelením práce a rôznou náročnosťou jednotlivých vetiev DFS prehľadávania.

5.3 Meranie MPI paralelizmu

Výsledky hybridnej MPI s OpenMP implementáciou sú uvedené v Tabuľke 2. Táto verzia kombinuje distribuovaný paralelizmus medzi procesmi a zdieľanú paralelizáciu pomocou OpenMP v rámci jednotlivých procesov.

Z výsledkov je zrejmé, že výkon MPI implementácie závisí od parametra `-c`, ktorý určuje veľkosť chunku odosielaného jednotlivým procesom. Príliš malé chunky vedú k častej komunikácii medzi procesmi a vyššej režii, zatiaľ čo príliš veľké chunky spôsobujú nevyvážené rozdelenie práce.

Najlepšie výsledky boli dosiahnuté pre stredné hodnoty chunku ($c = 10$), avšak rozdiely medzi jednotlivými hodnotami nie sú výrazné. Zároveň je viditeľné, že zvýšenie počtu vlákien v rámci jedného procesu (`t`) má len zanedbateľný pozitívny vplyv na výkon.

MPI implementácia bola testovaná s konfiguráciou 4 procesov (1 master + 3 slaves), čo zodpovedá odporúčaniam zadania a dostupnej architektúre klastra.

Typ	vlákna (`-t`)	chunk (`-c`)	hĺbka (`-l`)	mapa5_15			mapa15_5			mapa9_9		
				Čas [s]	S	E	Čas [s]	S	E	Čas [s]	S	E
MPI	2	4	4	9.466	18.05	2.26	8.125	24.27	3.03	7.055	65.56	8.19
	2	6	4	10.250	16.67	2.08	10.277	19.19	2.40	6.679	69.25	8.66
	2	8	4	10.865	15.73	1.97	10.446	18.88	2.36	6.506	71.09	8.89
	2	10	4	9.431	18.12	2.26	11.860	16.63	2.08	5.649	81.87	10.23
	4	4	4	6.676	25.59	1.60	4.283	46.05	2.88	6.837	67.65	4.23
	4	6	4	6.787	25.17	1.57	5.659	34.85	2.18	6.219	74.37	4.65
	4	8	4	6.751	25.31	1.58	4.419	44.63	2.79	5.874	78.74	4.92
	4	10	4	3.439	49.68	3.11	5.008	39.38	2.46	4.771	96.94	6.06
	6	4	4	9.614	17.77	0.74	5.749	34.31	1.43	7.069	65.43	2.73
	6	6	4	4.702	36.34	1.51	6.247	31.57	1.32	6.558	70.52	2.94
	6	8	4	9.229	18.51	0.77	4.120	47.87	1.99	6.257	73.92	3.08
	6	10	4	3.732	45.78	1.91	5.394	36.56	1.52	5.348	86.48	3.60
	8	4	4	9.519	17.95	0.56	5.498	35.87	1.12	7.122	64.94	2.03
	8	6	4	4.624	36.95	1.15	5.926	33.28	1.04	6.568	70.42	2.20
	8	8	4	9.333	18.31	0.57	3.663	53.84	1.68	6.357	72.75	2.27
	8	10	4	3.542	48.24	1.51	5.403	36.50	1.14	5.285	87.51	2.73
	12	4	4	9.370	18.23	0.38	5.814	33.92	0.71	7.092	65.21	1.36
	12	6	4	4.632	36.89	0.77	5.952	33.14	0.69	6.498	71.17	1.48
	12	8	4	9.203	18.57	0.39	3.746	52.65	1.10	6.335	73.01	1.52
	12	10	4	3.573	47.82	1.00	4.879	40.42	0.84	5.301	87.25	1.82

Tabuľka 2: MPI + OpenMP implementácia – čas, zrýchlenie a efektivita vzhľadom na sekvenčnú verziu. Efektivita je počítaná pre $p = 4t$.

6 Záver

V rámci práce boli implementované a porovnané viaceré paralelné prístupy k riešeniu problému skladania quatromín, konkrétne taskový a dátový paralelizmus pomocou OpenMP a hybridná MPI + OpenMP verzia. Na základe nameraných výsledkov sa ukazuje, že tasková paralelizácia dosahuje na meraných inštanciach najlepšie výkonnostné výsledky.

Dátová paralelizácia je naopak jednoduchšia a stabilnejšia, avšak menej efektívna. Jej nevýhodou je horšie vyváženie záťaže medzi vláknami, keďže jednotlivé úlohy majú rôznu výpočtovú náročnosť.

Hybridná MPI implementácia poskytuje konkurencieschopný výkon, pričom jej hlavnou výhodou je schopnosť distribuovať výpočet medzi viacero procesov. Na druhej strane je jej efektivita ovplyvnená nastavením parametrov, napríklad veľkosťou chunku, ktorá má vplyv na vyváženie práce a komunikačnú réžiu.

Efektivita paralelizácie však s rastúcim počtom vlákien klesá, čo je spôsobené rastúcou režijnou záťažou a synchronizáciou medzi vláknami. V niektorých konfiguráciách bolo pozorované zrýchlenie väčšie než počet použitých výpočtových jadier.

Tento jav však neinterpretujeme ako klasickú hardvérovú superlinearitu, ale ako dôsledok charakteru algoritmu typu Branch-and-Bound, kde paralelné prehľadávanie môže viesť k skoršiemu nájdeniu kvalitného riešenia a následne k intenzívnejšiemu prerezávaniu stavového priestoru.

V niektorých konfiguráciách vychádza vypočítaná efektivita väčšia než 1. Tento jav je spôsobený tým, že

pri algoritme typu Branch-and-Bound paralelná verzia nemusí prehľadávať rovnaký počet stavov ako sekvenčná verzia. Nejde teda o klasickú hardvérovú efektivitu, ale o dôsledok odlišného priebehu prehľadávania stavového priestoru.

Literatúra

[1] Langr, D., Šimeček, I., Šoch, M.: *Měření, vyhodnocení a sepsání zprávy*. FIT ČVUT, 2026.

[2] OpenMP Architecture Review Board: *OpenMP Application Programming Interface*. https://www.openmp.org/

[3] Message Passing Interface Forum: *MPI: A Message-Passing Interface Standard*. https://www.mpi-forum.org/

[4] SLURM Workload Manager: *SLURM Documentation*. https://slurm.schedmd.com/

[5] NI-PDP Course Materials: *Paralelní a distribuované programování*. FIT ČVUT.